Assignment Report on

Application Optimization, Scalability and Reliability

Submitted By	
Student Name:	Shahan Uz Zaman
Roll Number:	CS-BTC24-23
Course / Subject Name	Summer internship on Cyber Security and Secure App Development

1. Objective

The objective of this assignment is to improve the existing GUI-based multi-client TCP chat application developed in Assignment 7. The application was not redesigned from scratch. Instead, the existing authentication, Sign Up, Login, chat, private messaging, session management, and security features were extended with improvements for scalability, reliability, connection management, configuration, and performance.

The application was designed to support multiple concurrent clients while handling connection failures and resource cleanup more effectively. The assignment also requires performance evaluation using 5, 8, and 10 concurrent clients.

2. Scalability Improvements

The server was optimized to support at least 10 concurrent clients without crashing. A controlled thread pool was introduced using ThreadPoolExecutor instead of creating an unlimited number of threads for every connection.

The maximum number of workers and clients is configurable. Network operations are also separated from critical shared-state operations to reduce unnecessary lock contention.

The scalability testing is performed with:

- 5 concurrent clients
- 8 concurrent clients
- 10 concurrent clients

The Mininet topology specified in the assignment is:

sudo mn --topo single,11

The network is verified using nodes, net, and pingall.

3. Reliability Improvements

Several reliability improvements were added to the Assignment 7 implementation.

- **Automatic disconnection detection:**
  The server detects disconnected clients and removes their sessions from the active-client list.
- **Resource cleanup:**
  Sockets, usernames, and client information are properly removed when a client disconnects or times out.
- **Automatic reconnection:**
  The GUI client attempts to reconnect when the TCP connection is unexpectedly lost. The number of attempts and delay are configurable.
- **Timeout handling:**
  Socket timeouts prevent the application from waiting indefinitely for network operations.
- **Graceful shutdown:**
  The server handles shutdown signals and closes active resources properly instead of terminating abruptly.

- **Exception handling:**

  Network-related exceptions such as connection reset, broken pipe, timeout, and socket errors are handled without crashing the complete application.

These improvements directly address the Assignment 8 requirements for connection management and reliability.

4. Configuration Management

Previously, several application parameters were directly defined in the source code. In Assignment 8, configurable values were moved to config.json.

The configuration file contains parameters such as:

- *Server host*
- *Server port*
- *Socket timeout*
- *Maximum clients*
- *Maximum worker threads*
- *Session timeout*
- *Login lockout duration*
- *Buffer size*
- *Maximum message length*

This makes the application easier to maintain and allows configuration changes without modifying the main Python source code.

The use of ***config.json*** satisfies the configuration-management requirement of This assignment .

5. Performance Evaluation

A separate benchmarking program was added to evaluate the optimized application. The performance test measures:

- Average communication delay
- Throughput
- CPU usage
- Memory usage
- Number of concurrent clients
- Errors during testing

The application is tested with 5, 8, and 10 clients. Performance results are stored in:

performance_results.csv

Graphs are generated for comparison of the baseline Assignment 7 implementation and the optimized Assignment 8 implementation.

The following graphs are generated:

- Delay comparison
- Throughput comparison
- CPU usage comparison
- Memory usage comparison

The assignment requires these results and graphs to be generated from actual experiments; therefore, the final report should contain the actual values obtained during the student's Mininet tests rather than estimated values.

The results can be used to determine how the optimized application behaves as the number of concurrent clients increases.

6. Wireshark Verification

Wireshark was used to verify normal TCP communication between the clients and server. The TCP traffic was filtered using:

tcp.port == 5000

The communication was observed during normal client-server operation, including connection establishment and chat communication.

Screenshots of the Wireshark capture are included in the screenshots/ directory. The captured packets demonstrate that the application continues to use TCP port 5000 after optimization.

The assignment specifically requires Wireshark verification of normal TCP communication.

7. Challenges Faced

The major challenges during implementation were managing multiple concurrent clients, safely removing disconnected clients, preventing resource leaks, and handling network failures without crashing the server.

Another challenge was managing GUI widgets during reconnection and logout. The client was improved so that destroyed Tkinter widgets are not accessed after the login or chat window changes.

Performance testing with increasing numbers of clients also required careful measurement so that the results represented actual application behavior.

8. Conclusion

This assignment successfully extended the Assignment 7 secure TCP chat application with improvements in scalability, reliability, connection management, configuration management, and performance evaluation.

The use of controlled thread management allows the server to support multiple concurrent clients more efficiently. Automatic reconnection, timeout handling, graceful shutdown, exception handling, and resource cleanup improve application reliability. Moving configurable parameters to config.json also improves maintainability.

The application is designed to support the required 5, 8, and 10-client experiments, with performance results saved in CSV format and graphs generated for analysis. The Assignment 8 implementation therefore provides a more scalable, reliable, and maintainable version of the original multi-client TCP application while preserving the functionality developed in Assignment 7.